Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory Name: A. Priyankaa Reg. No.: 24011101001	AY: 2026–27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:
- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:
- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:
- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:
- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:
- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	**Size**	**Purpose**	**Applications**
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output
First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose
Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$= 448$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.
Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.
Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.
Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

$$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense \rightarrow Softmax$$

Train for
- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7
Evaluate
- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9. Source Code

The source code and implementation are available in this GitHub repository:
GitHub Repository

10. Mandatory Plots

1. Sample CIFAR-10 Images

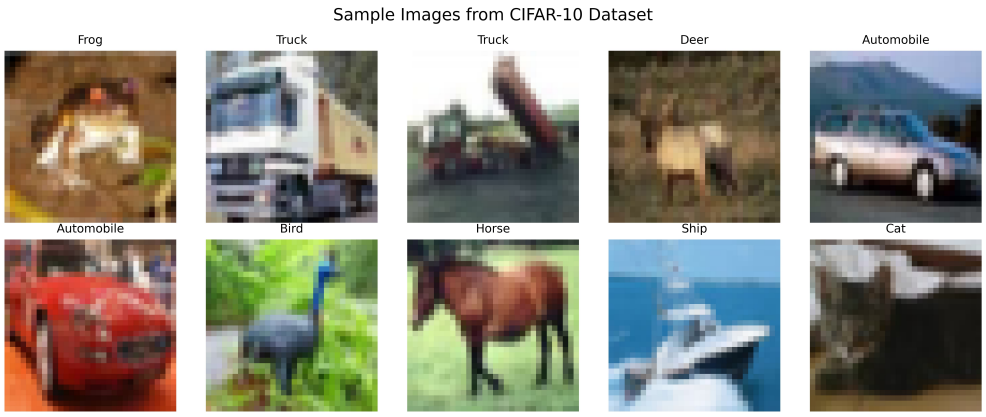

Figure 1: Ten sample images from the CIFAR-10.

Interpretation: The image shows different CIFAR-10 classes such as animals, vehicles, and other objects. The classes have different shapes, colours and patterns.

2. Class Distribution

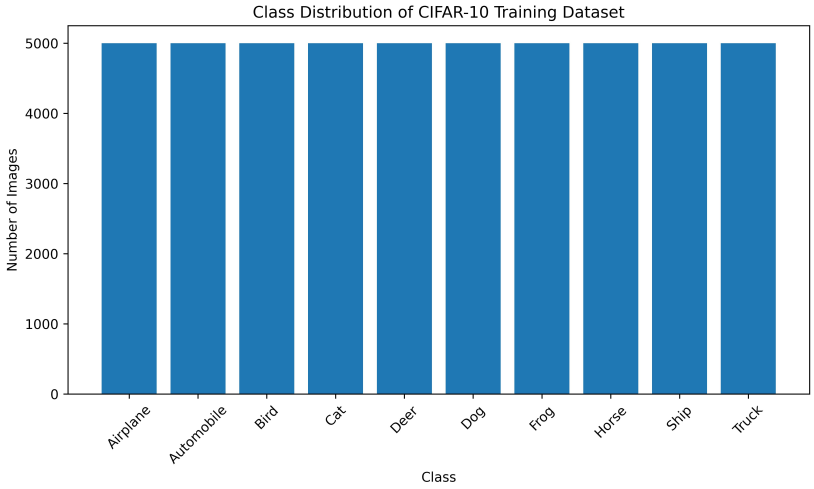

Figure 2: Class distribution across the ten CIFAR-10 categories in the training set.

Interpretation: There are 5,000 training images in each of the ten classes. This shows that the CIFAR-10 training set is balanced as each class has an equal number of images.

3. Training Accuracy and Validation Accuracy

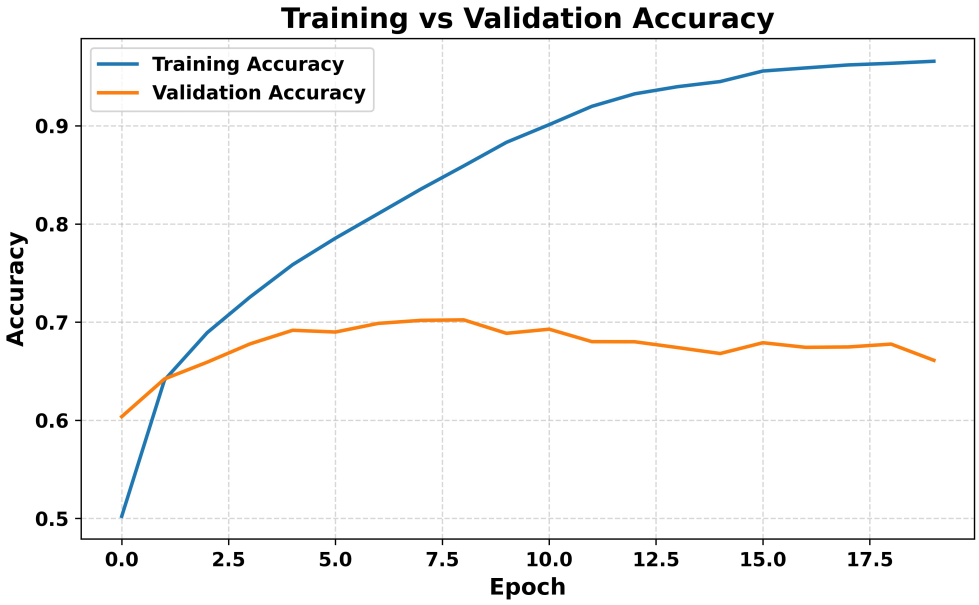

Figure 3: Training and validation accuracy across 20 epochs.

Interpretation: The training accuracy increases as the epochs increase and reaches 96.94% by the end. The validation accuracy improves at first and reaches around 70%, but then stays almost the same. This shows that the model is learning the training data much better than the unseen data.

4. Training Loss and Validation Loss

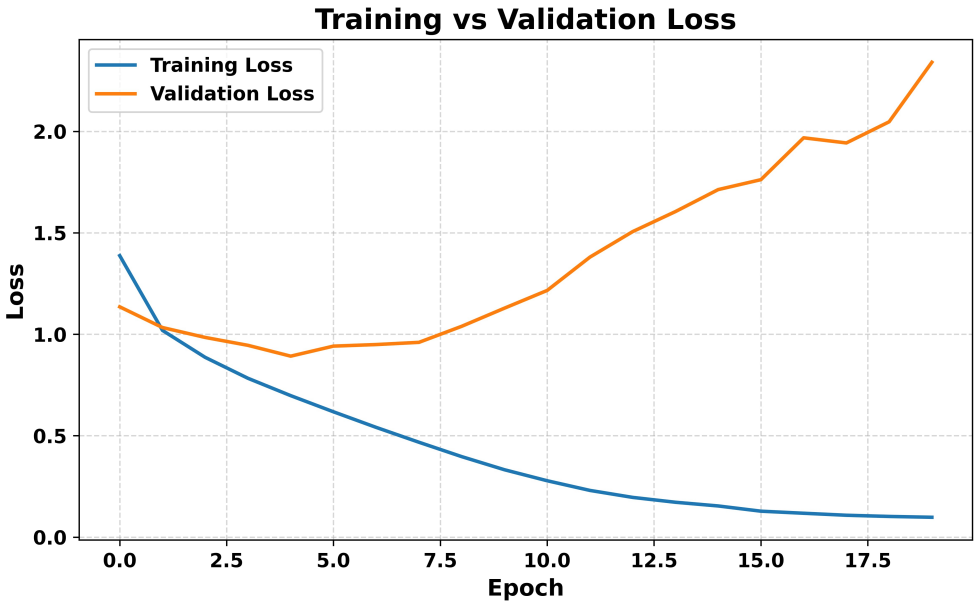

Figure 4: Training and validation loss across 20 epochs.

Interpretation: The training loss continues to fall while the model makes the training images better. The validation loss continues to fall initially but then begins to increase after the first few epochs until reaching a high value. This indicates the model is continually improving its use with the training images but is getting worse with unseen images.

5. Feature Maps

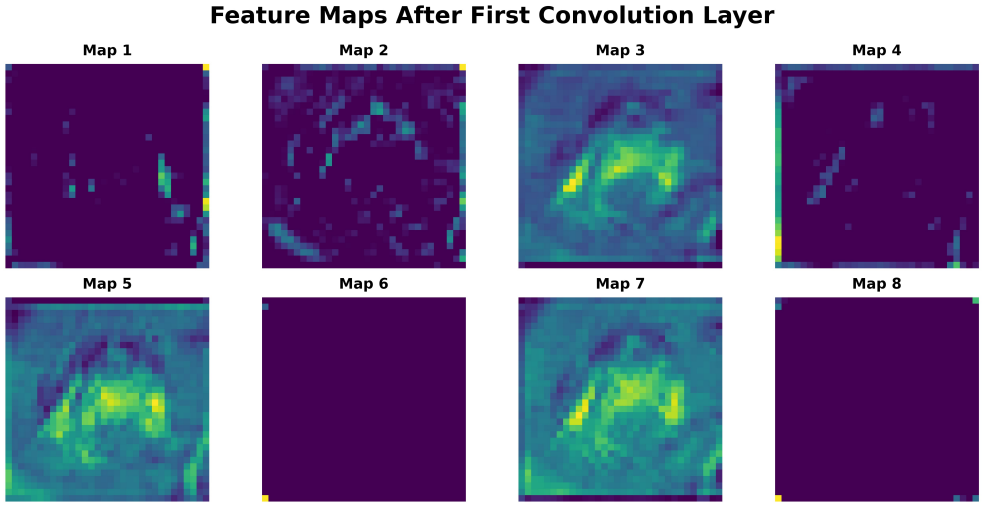

Figure 5: Feature maps generated after the first convolution layer.

Interpretation: The eight feature maps look different because each filter is identifying different features within the input image. Some maps show stronger areas around certain shapes and edges while others

have very little activation. This shows that different filters are learning different visual features from the input.

6. Confusion Matrix

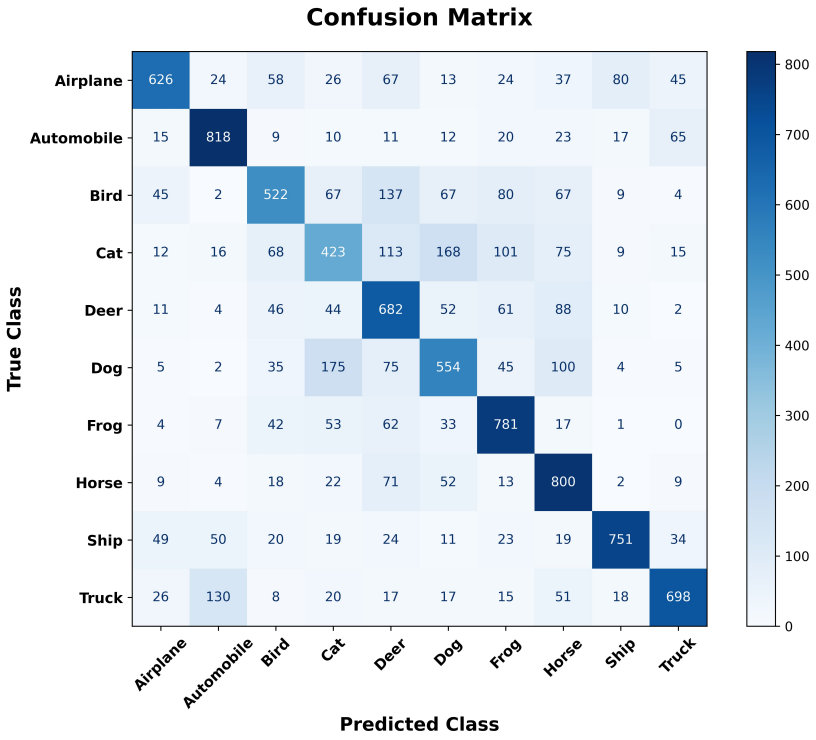

Figure 6: Confusion matrix of the CNN model on the CIFAR-10 test dataset.

Interpretation: Most of the correct predictions fall along the diagonal with classes such as Automobile, Frog, Horse and Ship exhibiting a relatively high number of correct predictions. There is evidence of confusion between similar classes such as Cat and Dog, there being many images that are classified as the other image. This indicates that the network finds objects that are visually similar more difficult to differentiate.

11. Results

1. Performance Metrics

Metric	Value
Training Accuracy	96.94%
Testing Accuracy	68.28%
Precision	0.6876
Recall	0.6828
F1-score	0.6834
Number of Parameters	545,098

12. Additional Exercises

1. Calculate the output size for a 64 × 64 image using a 5×5 kernel with stride 2 and padding 2.

Using,

$$\text{Output Size} = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

$$= \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 = 32$$

So the output size is:

$$\boxed{32 \times 32}$$

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

RGB has 3 input channels. Therefore,

$$(3 \times 3 \times 3 + 1) \times 64 = 1792$$

The number of trainable parameters is:

$$\boxed{1792}$$

3. Compare ReLU and Sigmoid activation functions.

ReLU is 0 when input is negative and it is unchanged when input is positive so this can be used in hidden layer of CNN. Sigmoid gives output from 0 to 1 and it is only useful when output like probability needed.

4. Replace Max Pooling with Average Pooling and compare the results. Both methods gave the same output size of $16 \times 16 \times 16$. Max Pooling gave an accuracy of 58.51%, slightly higher than the 57.80% by Average Pooling. Max Pooling, as it chooses the highest number from each region, preserves all necessary features for classification better than Average Pooling.

5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

Adding more filters enables the CNN to learn more features from the images. But it also increases the time of computation and training. And adding more filters does not mean higher accuracy.

13. Discussion

1. Why is convolution preferred over fully connected layers for images?

The convolution operation is applied to a small part of the image, preserving the spatial information. It requires less parameters than a fully connected layer since the same filter is shared over the whole image.

2. How does stride affect the feature map size?

Stride decides how far the filter moves each time. A larger stride gives a smaller feature map, while stride 1 keeps more of the original image information.

3. What is the role of padding?

Padding adds extra pixels around the edges of an image. It helps maintain the size of the feature map and also allows the border areas to be included in the convolution.

4. Why is pooling used?

Pooling reduces the size of the feature maps by reducing the number of calculations. It also preserves the important features while discarding some irrelevant details.

5. How do feature maps represent image characteristics?

Different filters look for different patterns in an image. The feature maps show where those patterns are found, such as edges, textures, or other visual details.

6. Why do CNNs require fewer parameters than MLPs? The same filter is applied to different parts of the image, as opposed to learning different weights for each position as in an MLP. This results in fewer parameters to learn.

14. Conclusion

A Convolutional Neural Network was implemented using TensorFlow/Keras and experimented on CIFAR-10 dataset to learn about convolution, kernel size, stride, padding, pooling, feature maps.

The last model consisted of two convolution layers, two Max Pooling layers, a Flatten, a dense and a Softmax output layer. It has been trained for 20 epochs using Adam with a batch size of 32.

The model had a training accuracy of 96.94% and a testing accuracy of 68.28%, using 545,098 trainable parameters. The fact that the training accuracy is higher than the testing accuracy indicates that the model has some overfitting and doesn‘t generalize well on new unseen images.

15. References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
3. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.